

CMSC6950 — Fall 2025

Homework #3

Due Date: Work must be submitted to GradeScope (for Problems 1-4) or GitHub (for Problem 5) by 5pm (St. John's time) on Tuesday, October 28.

1. (10 points) Find a public repository for a scientific software package on GitHub, and write 2-3 paragraphs explaining how it does or does not conform to the expectations of “good code” discussed in class. Be sure to provide a link to the GitHub repo and concrete examples of the properties of good code discussed in class.
2. (5 points) Use `assert` statements to write a python function that returns the value of $\cot(x) = \cos(x)/\sin(x)$, but only if $-\pi < x < \pi$ and $x \neq 0$. For values of x with $|x| \geq \pi$, your code should print a warning about the range of the function. For $x = 0$, it should print a separate warning about $\cot(x)$ not being defined for this value of x . Hand in a transcript of your Jupyter notebook, showing the definition of your function and the output of testing it for values of x outside of the range, at 0, and valid values. You can use the `math` module to compute the value of $\cot(x)$ using the above formula.
3. (5 points) Use `try` and `except` statements to write a python function that returns the value of $\csc(x) = 1/\sin(x)$ when it is well-defined, but prints a suitable error message when $\sin(x) = 0$ and also when the input is not a real number. Hand in a transcript of your Jupyter notebook, showing the definition of your function, and the output of testing it for values of x where $\csc(x)$ is well-defined, values where $\sin(x) = 0$, and invalid input types for x .
4. (10 points) Consider the two decorators given by

```
class add_constant(object):
    def __init__(self, constant):
        self.constant = constant
    def __call__(self, f):
        def result(x):
            return f(x)+self.constant
        return result

def x_times_f(f):
    def result(x):
        return x*f(x)
    return result
```

which can be called with syntax

```
@add_constant(2)
def x_plus_two(x):
    return x

@x_times_f
def x_squared(x):
    return x
```

- (a) Explain the difference between the two functions

```
@x_times_f
@add_constant(4)
def f1(x):
    return x
```

```
@add_constant(4)
@x_times_f
def f2(x):
    return x
```

What mathematical functions are being computed in the two cases?

- (b) Consider the code

```
@add_constant(1)
@x_times_f
@add_constant(3)
def q(x):
    return x
```

What mathematical function, $q(x)$, does this compute?

- (c) Compose the two decorators to write a function that evaluates the polynomial $p(x) = x^3 + 5x^2 - 2x + 3$. Hand in a transcript of your Jupyter notebook, showing your function definition and some tests that it produces the correct values. No points will be given for functions that evaluate $p(x)$ directly, only for functions that use the decorators.

5. (30 points) Go to <https://classroom.github.com/a/4b1qnMV5> and accept the “HW3 Pytest” assignment. Clone the repository, read the instructions in the README.md file, complete the assignment, and push your code to the repo to be graded. Nothing needs to be handed in to GradeScope for this problem.